07th June 2026



Università degli Studi di Padova

Università Degli Studi di Padova

Cyber-Physical Systems and IoT Security 2025/2026

Mid-Course Project Report

Reference paper: Error Handling of In-vehicle Networks Makes Them Vulnerables

Student: Valentina Beni
Student ID: 2155350

Contents

1 Objectives **2**

2 Setup **2**

3 Experiments **2**

4 Results **4**

1 Objectives

This project presents a simulation designed to reproduce a type of Denial of Service attack known as a "bus-off attack", which affects in-vehicle networks such as the CAN bus. The experiment involve modeling the attack's exploitation of error-handling mechanisms to disconnect uncompromised ECUs and testing it on a CAN bus prototype.
The project was implemented on a PC using Python, unlike the original paper which used real hardware. Therefore it was necessary to develop both the code to simulate components like the ECUs and the CAN Bus, as well as the code to perform the attack described in the paper.

2 Setup

While the original paper implemented the attack using real hardware components, this project reproduces it in a simulated environment using Python, modeling both an ECU and a CAN bus before executing the attack.
In order to execute the code, it is recommended to create a virtual environment and install the libraries listed in the file `requirements.txt`, following the commands:

```
python -m venv my_venv
.\my_venv\Scripts\activate
pip install -r requirements.txt
```

Once the dependencies are installed, it is sufficient to execute the `main.py` script to run the simulation and analyze the results.

```
py main.py
```

3 Experiments

The project folder is composed by three classes used to simulate the interaction between the CAN Bus and the ECU, a `utils.py` file and the `main.py` file.
The following files are used to simulate the attack:

- **Node.py**: Represents an ECU node in the CAN bus simulation;
- **Bus.py**: Represents the CAN bus in the simulation;
- **Message.py**: Represents a CAN message in the simulation;
- **utils.py**: Used to plot the results of the simulation.

The Message class represents the data transmitted by an ECU over the CAN bus. It is a highly simplified representation of a data packet, designed exclusively for this simulation and therefore does not include the full set of features of a real-world message.
Each message has an associated ECU ID (the sender), a message ID, and a data payload. The class provides methods to convert the message to an array format for logging and to retrieve its components (ECU ID, message ID, and data).

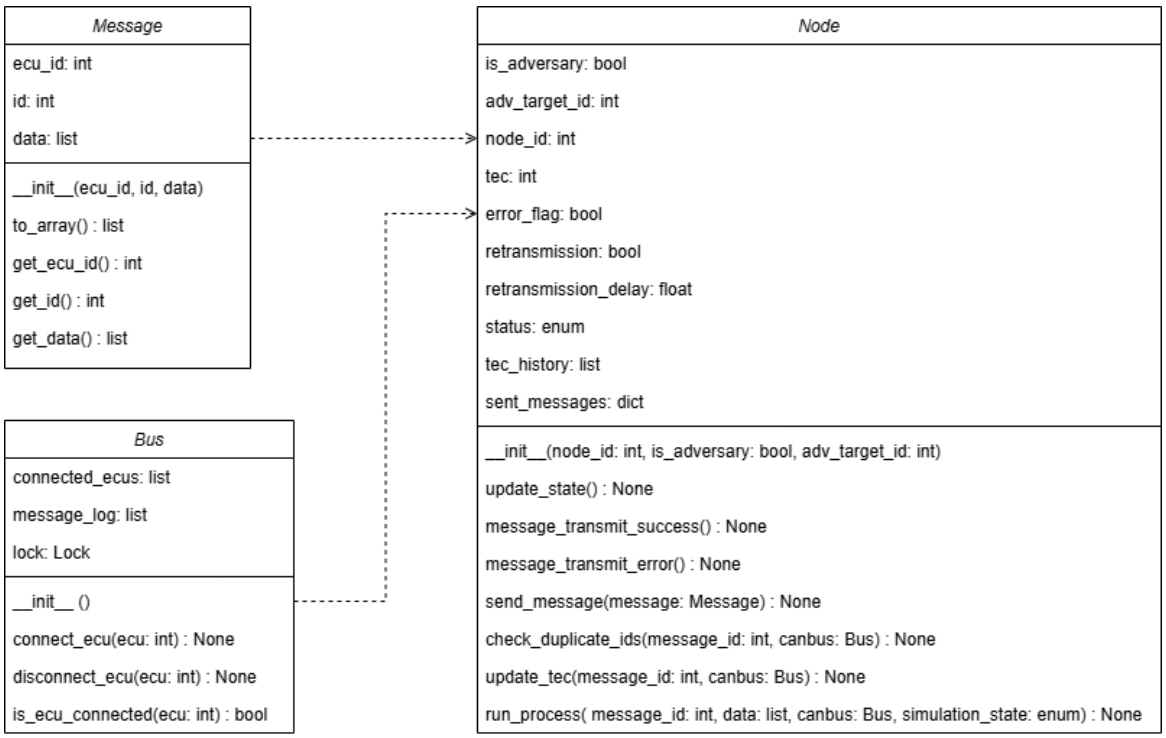


Figure 1: Class diagram

The Bus class maintains a list of connected ECUs and a log of all messages transmitted on the bus. It provides methods for connecting and disconnecting ECUs, checking if an ECU is connected and logging messages. The bus uses a lock to ensure thread-safe access to the list of connected ECUs and the message log, allowing multiple nodes to interact with the bus concurrently without data corruption. Like the Message class, this is a simplified class that only includes the features necessary for this simulation to function.
Lastly, the Node class simulate an ECU node. Each node can be a normal ECU or an adversary ECU that manipulates its error state to disrupt communication. The node maintains its own Transmit Error Counter (TEC) and updates it based on the success or failure of message transmissions. It also checks for duplicate message IDs on the bus and adjusts its error state accordingly. The node runs a process that simulates sending messages, checking for duplicates, and updating its TEC in a loop until the simulation is stopped or the node goes bus-off. Each node is created with the state "Error Active", the state of a node is updated at each change of the TEC counter to reflect the passage to the "Error Passive" or "Bus-off" state.
The core of the simulation is the `run_process` function, wich implements the algorithm for each ECU node in the CAN bus simulation. It runs in a loop while the simulation is active, the node remains connected, and at least two ECUs are present on the bus. If the node enters the Bus-off state, it disconnects from the bus and stops the simulation. The function uses a threading barrier to synchronize message transmission between nodes. It sends a message (either a new one or a retransmission if a duplicate was detected) and then checks for duplicate message IDs on the bus. Afterward, it updates the TEC based on whether the node is an adversary or a normal ECU, adjusting the error state accordingly.
This process simulates the real-time behavior of ECUs on a CAN bus, including error handling and message retransmission.

4 Results

The simulation successfully replicated the bus-off attack dynamics described in the reference paper, demonstrating how an adversarial ECU can exploit the error-handling mechanisms of the CAN bus to force a legitimate ECU into the Bus-off state.
As shown in the logs and Figure 2, the Transmit Error Counter (TEC) for both ECUs initially increases by 8 as they retransmit conflicting messages.

```
The ECU 1 has been connected to the CAN bus
The ECU 2 has been connected to the CAN bus
ECU 2 sent message: ID=1 with data=[1, 1, 0] -> TEC = 0
ECU 1 sent message: ID=1 with data=[1, 1, 1] -> TEC = 0
ECU 1 is retransmitting message ID=1 with data=[1, 1, 1] -> TEC = 8
ECU 2 is retransmitting message ID=1 with data=[1, 1, 0] -> TEC = 8
ECU 1 is retransmitting message ID=1 with data=[1, 1, 1] -> TEC = 16
ECU 2 is retransmitting message ID=1 with data=[1, 1, 0] -> TEC = 16
...
```

Once the TEC of the victim ECU (ECU 1) reaches the Error Passive threshold (128), the attacker (ECU 2) begins to manipulate its error state by sending messages with non-dominant bits, causing the victim's TEC to rise rapidly while its own TEC decreases.
The victim TEC increase is given by a failed transmission (+8) followed by a successful retransmission (-1) wich cause the TEC value to increase by 7, while the TEC value of the attacker decrease by 1. This asymmetry ultimately forces the victim into the Bus-off state, effectively disconnecting it from the bus, while the attacker remains in an Error Active state.

```
...
ECU 1 is retransmitting message ID=1 with data=[1, 1, 1] -> TEC = 240
ECU 1 is retransmitting message ID=1 with data=[1, 1, 1] -> TEC = 247
The ECU 1 has been disconnected from the CAN bus
```

A comparison between the simulation result and the paper's experimental results in Figure 3 confirms that the implemented model accurately captures the attack's behavior. While the rates of TEC increase differ slightly, due to simulation-specific parameters such as message delay times, the overall trends align closely with the theoretical and practical findings of the original study.
This validation demonstrates that the Python-based simulation, despite its simplified design, effectively mirrors the real-world behavior of CAN bus error handling under adversarial conditions.

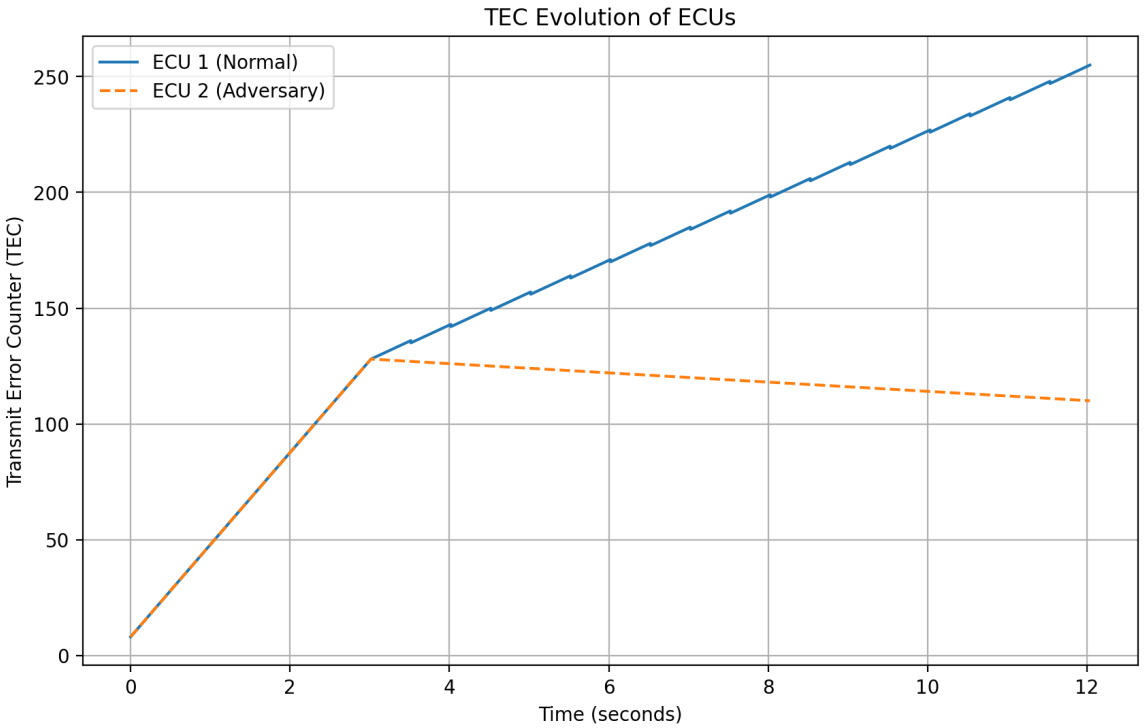


Figure 2: Simulation results

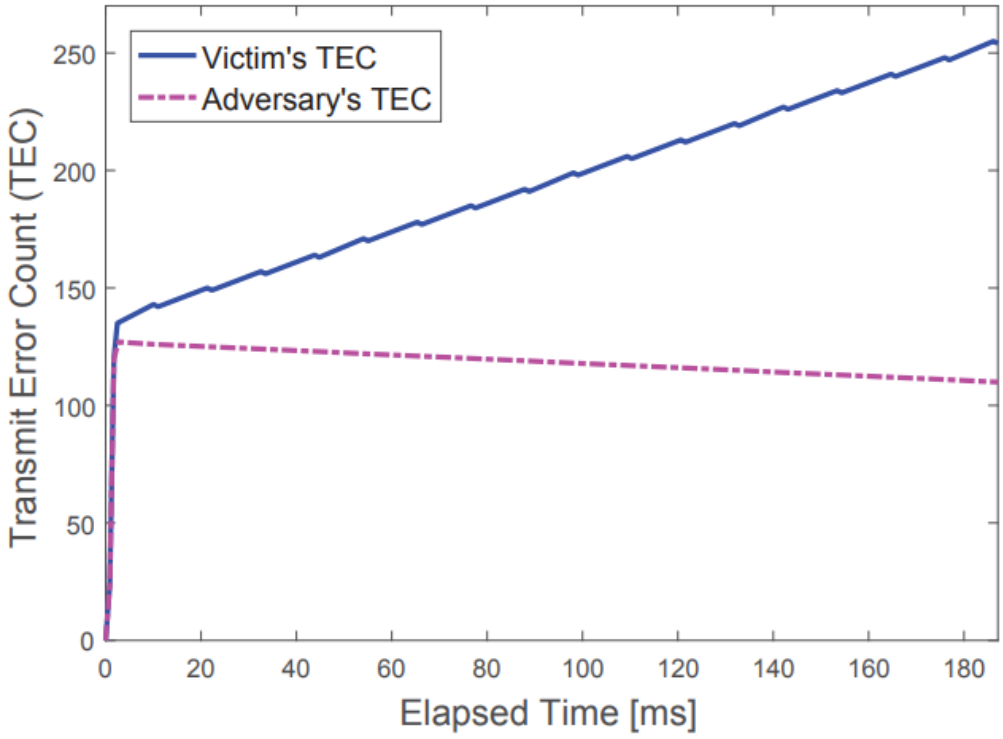


Figure 3: Paper results